PROJECT' (PROJECT-NO, PROJECT-DESCRIPTION, PROJECT-MANAGER-NO)
PART-PROJECT' (PART-NO, PROJECT-NO, QUANTITY-COMMITTED).

Using the entity-relationship diagram in Figure 11, the following entity and relationship relations can be easily derived:

entity relations	PART''(PART-NO, PART-DESCRIPTION, QUANTITY-ON-HAND) PROJECT''(PROJECT-NO, PROJECT-DESCRIPTION) EMPLOYEE''(EMPLOYEE-NO)
relationship relations	PART-PROJECT''(PART/PART-NO, PROJECT/PROJECT-NO, QUANTITY-COMMITTED) PROJECT-MANAGER''(PROJECT/PROJECT-NO, MANAGER/EMPLOYEE-NO).

The role names of the entities in relationships (such as MANAGER) are indicated. The entity relation names associated with the PKs of entities in relationships and the value set names have been omitted.

Note that in the example above, entity/relationship relations are similar to the 3NF relations. In the 3NF approach, PROJECT-MANAGER-NO is included in the relation PROJECT' since PROJECT-MANAGER-NO is assumed to be functionally dependent on PROJECT-NO. In the entity-relationship model, PROJECT-MANAGER-NO (i.e. EMPLOYEE-NO of a project manager) is included in a relationship relation PROJECT-MANAGER since EMPLOYEE-NO is considered as an entity PK in this case.

Also note that in the 3NF approach, changes in functional dependencies of data may cause some relations not to be in 3NF. For example, if we make a new assumption that one project may have more than one manager, the relation PROJECT' is no longer a 3NF relation and has to be split into two relations as PROJECT'' and PROJECT-MANAGER''. Using the entity-relationship model, no such change is necessary. Therefore, we may say that by using the entity-relationship model we can arrange data in a form similar to 3NF relations but with clear semantic meaning.

It is interesting to note that the decomposition (or transformation) approach described above for normalization of relations may be viewed as a bottom-up approach in database design.[4] It starts with arbitrary relations (level 3 in Figure 1) and then uses some semantic information (functional dependencies of data) to transform them into 3NF relations (level 2 in Figure 1). The entity-relationship model adopts a top-down approach, utilizing the semantic information to organize data in entity/relationship relations.

4.2 The Network Model

4.2.1 Semantics of the Data-Structure Diagram. One of the best ways to explain the network model is by use of the *data-structure diagram* [3]. Figure 16(a) illustrates a data-structure diagram. Each rectangular box represents a record type.

[4] Although the decomposition approach was emphasized in the relational model literature, it is a procedure to obtain 3NF and may not be an intrinsic property of 3NF.

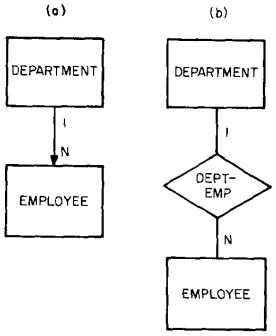

Fig. 16. Relationship DEPARTMENT–EMPLOYEE
(a) data structure diagram
(b) entity-relationship diagram

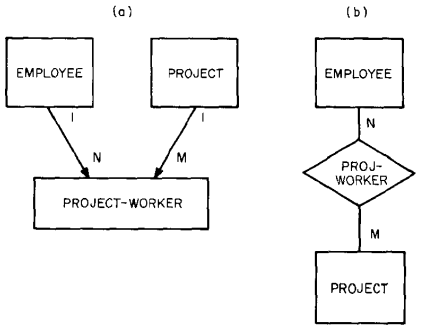

Fig. 17. Relationship PROJECT-WORKER
(a) data structure diagram
(b) entity-relationship diagram

The arrow represents a data-structure-set in which the DEPARTMENT record is the *owner-record*, and one owner-record may own n ($n = 0, 1, 2, \ldots$) *member-records*. Figure 16(b) illustrates the corresponding entity-relationship diagram. One might conclude that the arrow in the data-structure diagram represents a relationship between entities in two entity sets. This is not always true. Figures 17(a) and 17(b) are the data-structure diagram and the entity-relationship diagram expressing the relationship PROJECT–WORKER between two entity types EMPLOYEE and PROJECT. We can see in Figure 17(a) that the relationship PROJECT–WORKER becomes another record type and that the arrows no longer represent relationships between entities. What are the real meanings of the arrows in data-structure diagrams? The answer is that an arrow represents an $1{:}n$ relationship between two *record* (not entity) types and also implies the existence of an access path from the owner record to the member records. The data-structure diagram is a representation of the organization of records (level 4 in Figure 1) and is not an exact representation of entities and relationships.

4.2.2 Deriving the Data-Structure Diagram. Under what conditions does an arrow in a data-structure diagram correspond to a relationship of entities? A close comparison of the data-structure diagrams with the corresponding entity-relationship diagrams reveals the following rules:

1. For $1{:}n$ binary relationships an arrow is used to represent the relationship (see Figure 16(a)).
2. For $m{:}n$ binary relationships a "relationship record" type is created to represent the relationship and arrows are drawn from the "entity record" type to the "relationship record" type (see Figure 17(a)).
3. For k-ary ($k \geq 3$) relationships, the same rule as (2) applies (i.e. creating a "relationship record" type).

Since DBTG [7] does not allow a data-structure-set to be defined on a single record type (i.e. Figure 18 is not allowed although it has been implemented in [13]), a "relationship record" is needed to implement such relationships (see

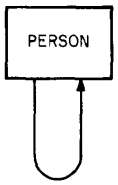

Fig. 18. Data-structure-set defined on the same record type

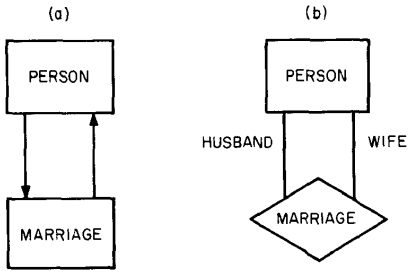

Fig. 19. Relationship MARRIAGE (a) data structure diagram (b) entity–relationship diagram

Figure 19(a)) [20]. The corresponding entity-relationship diagram is shown in Figure 19(b).

It is clear now that arrows in a data-structure diagram do not always represent relationships of entities. Even in the case that an arrow represents a $1{:}n$ relationship, the arrow only represents a unidirectional relationship [20] (although it is possible to find the owner-record from a member-record). In the entity-relationship model, both directions of the relationship are represented (the roles of both entities are specified). Besides the semantic ambiguity in its arrows, the network model is awkward in handling changes in semantics. For example, if the relationship between DEPARTMENT and EMPLOYEE changes from a $1{:}n$ mapping to an $m{:}n$ mapping (i.e. one employee may belong to several departments), we must create a relationship record DEPARTMENT–EMPLOYEE in the network model.

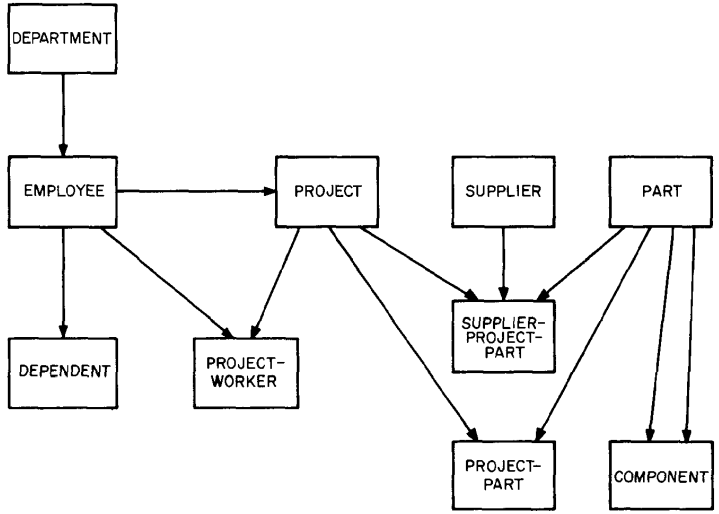

Fig. 20. The data structure diagram derived from the entity–relationship diagram in Fig. 11

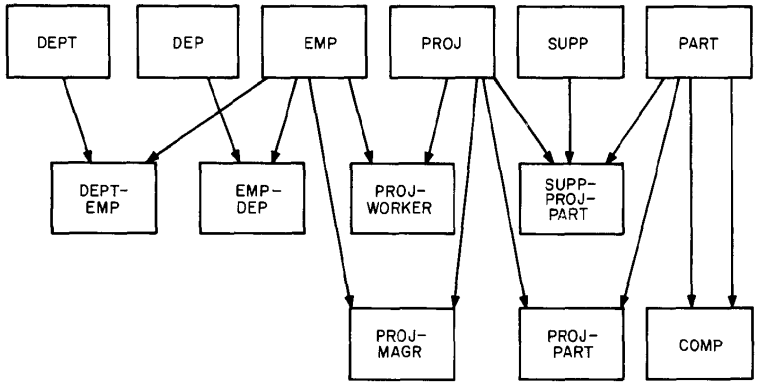

Fig. 21. The "disciplined" data structure diagram derived from the entity-relationship diagram in Fig. 11

In the entity-relationship model, all kinds of mappings in relationships are handled uniformly.

The entity-relationship model can be used as a tool in the structured design of databases using the network model. The user first draws an entity-relationship diagram (Figure 11). He may simply translate it into a data-structure diagram (Figure 20). using the rules specified above. He may also follow a discipline that every entity or relationship must be mapped onto a record (that is, "relationship records" are created for all types of relationships no matter that they are $1:n$ or $m:n$ mappings). Thus, in Figure 11, all one needs to do is to change the diamonds to boxes and to add arrowheads on the appropriate lines. Using this approach three more boxes—DEPARTMENT–EMPLOYEE, EMPLOYEE–DEPENDENT, and PROJECT–MANAGER—will be added to Figure 20 (see Figure 21). The validity constraints discussed in Sections 3.3–3.5 will also be useful.

4.3 The Entity Set Model

4.3.1 The Entity Set View. The basic element of the entity set model is the entity. Entities have names (*entity names*) such as "Peter Jones", "blue", or "22". Entity names having some properties in common are collected into an *entity-name-set*, which is referenced by the *entity-name-set-name* such as "NAME", "COLOR", and "QUANTITY".

An entity is represented by the entity-name-set-name/entity-name pair such as NAME/Peter Jones, EMPLOYEE-NO/2566, and NO-OF-YEARS/20. An entity is described by its association with other entities. Figure 22 illustrates the entity set view of data. The "DEPARTMENT" of entity EMPLOYEE-NO/2566 is the entity DEPARTMENT-NO/405. In other words, "DEPARTMENT" is the role that the entity DEPARTMENT-NO/405 plays to describe the entity EMPLOYEE-NO/2566. Similarly, the "NAME", "ALTERNATIVE-NAME", or "AGE" of EMPLOYEE-NO/2566 is "NAME/Peter Jones", "NAME/Sam Jones", or "NO-OF-YEARS/20", respectively. The description of the entity EMPLOYEE-

NO/2566 is a collection of the related entities and their roles (the entities and roles circled by the dotted line). An example of the *entity description* of "EMPLOYEE–NO/2566" (in its full-blown, unfactored form) is illustrated by the set of role-name/entity-name-set-name/entity-name triplets shown in Figure 23. Conceptually, the entity set model differs from the entity-relationship model in the following ways:

(1) In the entity set model, everything is treated as an entity. For example, "COLOR/BLACK" and "NO–OF–YEARS/45" are entities. In the entity-relationship model, "blue" and "36" are usually treated as values. Note treating values as entities may cause semantic problems. For example, in Figure 22, what is the difference between "EMPLOYEE–NO/2566", "NAME/Peter Jones", and "NAME/Sam Jones"? Do they represent different entities?

(2) Only binary relationships are used in the entity set model,[5] while n-ary relationships may be used in the entity-relationship model.

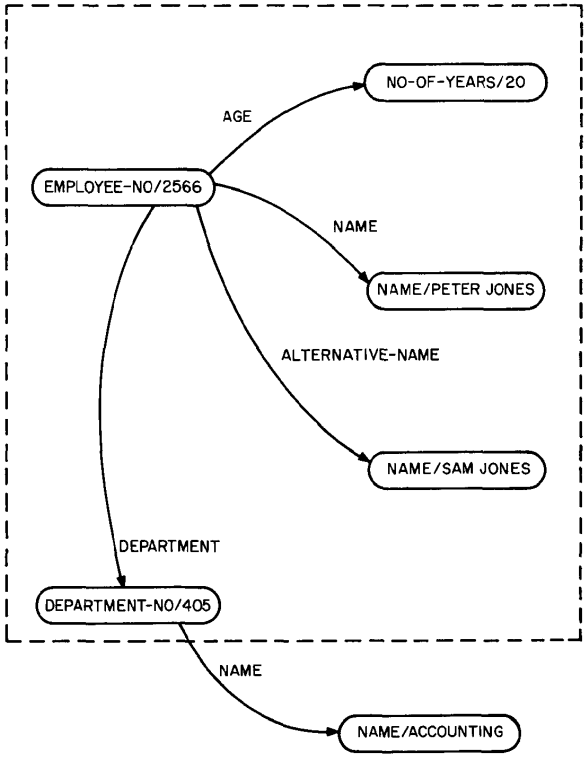

Fig. 22. The entity-set view

[5] In DIAM II [24], n-ary relationships may be treated as special cases of identifiers.

THE ENTITY-RELATIONSHIP MODEL TERMINOLOGY	ATTRIBUTE OR ROLE	VALUE SET	VALUE
THE ENTITY SET MODEL TERMINOLOGY	"ROLE-NAME"	"ENTITY-NAME-SET-NAME"	"ENTITY-NAME"
	IDENTIFIER	EMPLOYEE-NO	2566
	NAME	NAME	PETER JONES
	NAME	NAME	SAM JONES
	AGE	NO-OF-YEARS	25
	DEPARTMENT	DEPARTMENT-NO	405

Fig. 23. An "entity description" in the entity-set model

4.3.2 Deriving the Entity Set View. One of the main difficulties in understanding the entity set model is due to its world view (i.e. identifying values with entities). The entity-relationship model proposed in this paper is useful in understanding and deriving the entity set view of data. Consider Figures 2 and 6. In Figure 2, entities are represented by e_i's (which exist in our minds or are pointed at with fingers). In Figure 6, entities are represented by values. The entity set model works both at level 1 and level 2, but we shall explain its view at level 2 (Figure 6). The entity set model treats all value sets such as NO-OF-YEARS as "entity-name-sets" and all values as "entity-names." The attributes become role names in the entity set model. For binary relationships, the translation is simple: the role of an entity in a relationship (for example, the role of "DEPARTMENT" in the relationship DEPARTMENT-EMPLOYEE) becomes the role name of the entity in describing the other entity in the relationship (see Figure 22). For n-ary ($n > 2$) relationships, we must create artificial entities for relationships in order to handle them in a binary relationship world.

ACKNOWLEDGMENTS

The author wishes to express his thanks to George Mealy, Stuart Madnick, Murray Edelberg, Susan Brewer, Stephen Todd, and the referees for their valuable sug-

gestions (Figure 21 was suggested by one of the referees). This paper was motivated by a series of discussions with Charles Bachman. The author is also indebted to E.F. Codd and M.E. Senko for their valuable comments and discussions in revising this paper.

REFERENCES

1. ABRIAL, J.R. Data semantics. In *Data Base Management*, J.W. Klimbie and K.L. Koffeman, Eds., North-Holland Pub. Co., Amsterdam, 1974, pp. 1–60.
2. BACHMAN, C.W. Software for random access processing. *Datamation 11* (April 1965), 36–41.
3. BACHMAN, C.W. Data structure diagrams. *Data Base 1*, 2 (Summer 1969), 4–10.
4. BACHMAN, C.W. Trends in database management—1975. Proc., AFIPS 1975 NCC, Vol. 44, AFIPS Press, Montvale, N.J., pp. 569–576.
5. BIRKHOFF, G., AND BARTEE, T.C. *Modern Applied Algebra*. McGraw-Hill, New York, 1970.
6. CHAMBERLIN, D.D., AND RAYMOND, F.B. SEQUEL: A structured English query language. Proc. ACM-SIGMOD 1974, Workshop, Ann Arbor, Michigan, May, 1974.
7. CODASYL. Data base task group report. ACM, New York, 1971.
8. CODD, E.F. A relational model of data for large shared data banks. *Comm. ACM 13*, 6 (June 1970), 377–387.
9. CODD, E.F. Normalized data base structure: A brief tutorial. Proc. ACM-SIGFIDET 1971, Workshop, San Diego, Calif., Nov. 1971, pp. 1–18.
10. CODD, E.F. A data base sublanguage founded on the relational calculus. Proc. ACM-SIGFIDET 1971, Workshop, San Diego, Calif., Nov. 1971, pp. 35–68.
11. CODD, E.F. Recent investigations in relational data base systems. Proc. IFIP Congress 1974, North-Holland Pub. Co., Amsterdam, pp. 1017–1021.
12. DEHENEFFE, C., HENNEBERT, H., AND PAULUS, W. Relational model for data base. Proc. IFIP Congress 1974, North-Holland Pub. Co., Amsterdam, pp. 1022–1025.
13. DODD, G.G. APL—a language for associate data handling in PL/I. Proc. AFIPS 1966 FJCC, Vol. 29, Spartan Books, New York, pp. 677–684.
14. ESWARAN, K.P., AND CHAMBERLIN, D.D. Functional specifications of a subsystem for data base integrity. Proc. Very Large Data Base Conf., Framingham, Mass., Sept. 1975, pp. 48–68.
15. HAINAUT, J.L., AND LECHARLIER, B. An extensible semantic model of data base and its data language. Proc. IFIP Congress 1974, North-Holland Pub. Co., Amsterdam, pp. 1026–1030.
16. HAMMER, M.M., AND MCLEOD, D.J. Semantic integrity in a relation data base system. Proc. Very Large Data Base Conf., Framingham, Mass., Sept. 1975, pp. 25–47.
17. LINDGREEN, P. Basic operations on information as a basis for data base design. Proc. IFIP Congress 1974, North-Holland Pub. Co., Amsterdam, pp. 993–997.
18. MEALY, G.H. Another look at data base. Proc. AFIPS 1967 FJCC, Vol. 31, AFIPS Press, Montvale, N.J., pp. 525–534.
19. NIJSSEN, G.M. Data structuring in the DDL and the relational model. In *Data Base Management*, J.W. Klimbie and K.L. Koffeman, Eds., North-Holland Pub. Co., Amsterdam, 1974, pp. 363–379.
20. OLLE, T.W. Current and future trends in data base management systems. Proc. IFIP Congress 1974, North-Holland Pub. Co., Amsterdam, pp. 998–1006.
21. ROUSSOPOULOS, N., AND MYLOPOULOS, J. Using semantic networks for data base management. Proc. Very Large Data Base Conf., Framingham, Mass., Sept. 1975, pp. 144–172.
22. RUSTIN, R. (Ed.). Proc. ACM-SOGMOD 1974—debate on data models. Ann Arbor, Mich., May 1974.
23. SCHMID, H.A., AND SWENSON, J.R. On the semantics of the relational model. Proc. ACM-SIGMOD 1975, Conference, San Jose, Calif., May 1975, pp. 211–233.
24. SENKO, M.E. Data description language in the concept of multilevel structured description: DIAM II with FORAL. In *Data Base Description*, B.C.M. Dougue, and G.M. Nijssen, Eds., North-Holland Pub. Co., Amsterdam, pp. 239–258.

25. SENKO, M.E., ALTMAN, E.B., ASTRAHAN, M.M., AND FEHDER, P.L. Data structures and accessing in data-base systems. *IBM Syst. J. 12*, 1 (1973), 30–93.
26. SIBLEY, E.H. On the equivalence of data base systems. Proc. ACM-SIGMOD 1974 debate on data models, Ann Arbor, Mich., May 1974, pp. 43–76.
27. STEEL, T.B. Data base standardization—a status report. Proc. ACM-SIGMOD 1975, Conference, San Jose, Calif., May 1975, pp. 65–78.
28. STONEBRAKER, M. Implementation of integrity constraints and views by query modification. Proc. ACM-SIGMOD 1975, Conference, San Jose, Calif., May 1975, pp. 65–78.
29. SUNDGREN, B. Conceptual foundation of the infological approach to data bases. In *Data Base Management*, J.W. Klimbie and K.L. Koffeman, Eds., North-Holland Pub. Co., Amsterdam, 1974, pp. 61–95.
30. TAYLOR, R.W. Observations on the attributes of database sets. In *Data Base Description*, B.C.M. Dougue and G.M. Nijssen, Eds., North-Holland Pub. Co., Amsterdam, pp. 73–84.
31. TSICHRITZIS, D. A network framework for relation implementation. In *Data Base Description*, B.C.M. Douge and G.M. Nijssen, Eds., North-Holland Pub. Co., Amsterdam, pp. 269–282.